
From Vibes to Verdicts

Building and Evaluating Agents That Take Action

Aarshay Jain

What we'll cover

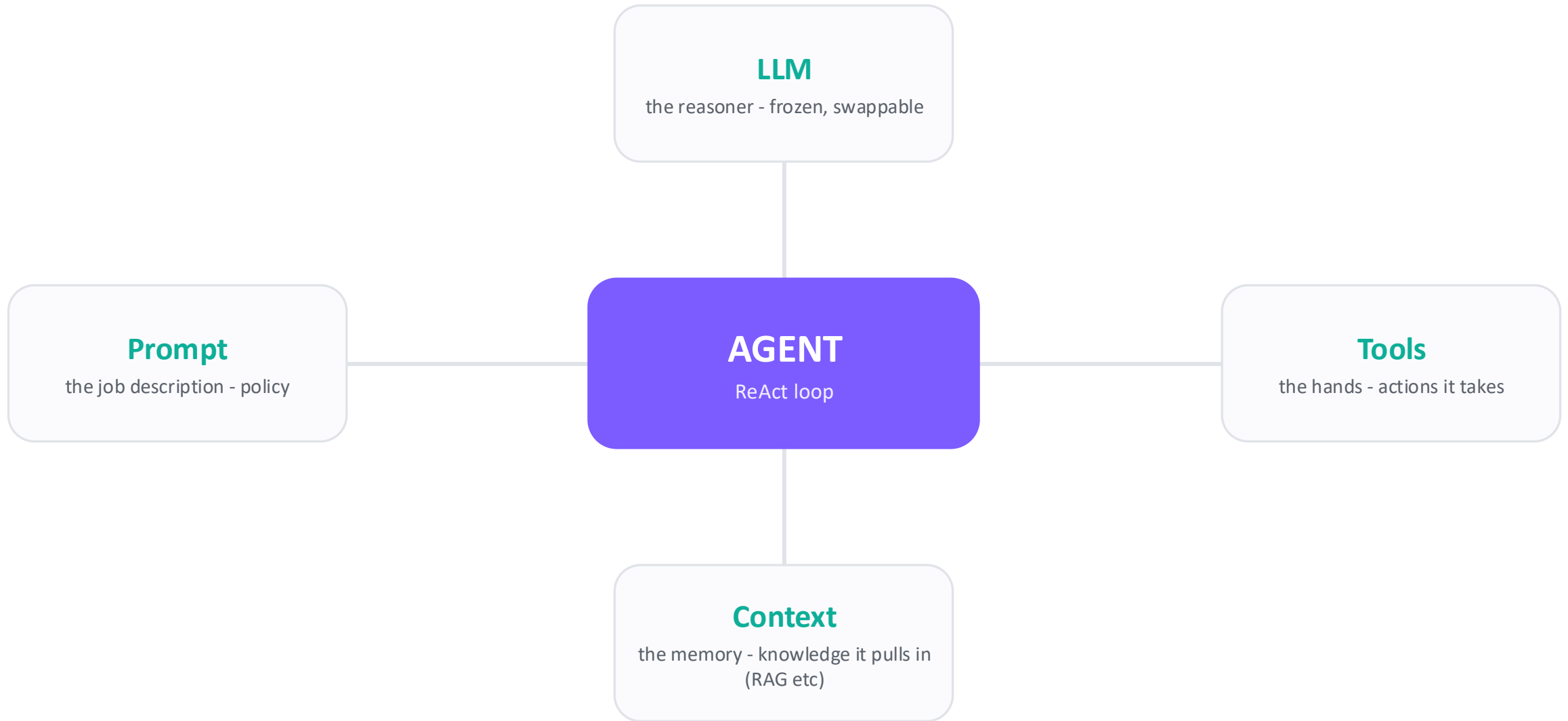
- 1 MLOps → AgentOps
- 2 Anatomy of an agent
- 3 Evaluating an agent
- 4 Agent architectures

MLOps ⇒ AgentOps

Axis	Traditional ML	Agentic LLM
What you ship	Trained weights	Prompt + tools + context + policy around a frozen model
Dev loop	Collect → train → test	Prompt → eval → trace → revise
Iteration unit	Features, hyperparams	Prompts, tools, policy (“context engineering”)
Evaluation	F1 / AUC on held-out set	LLM-as-judge over multi-turn trajectories
Failure modes	Overfitting, data drift	Hallucination, tool misuse, cascading errors
Observability	Metrics dashboards	Full trace/span trees of reasoning + tool calls

AGENT ANATOMY

Answering uses prompt + context. Acting needs tools.



OUR EXPERIMENT

The airline agent (tau2-bench)

dataset · github.com/sierra-research/tau2-bench

PROMPT

From the airline **policy** - sections:

info · **booking** · **modify**

cancel · **compensation**

each section = rules the agent must apply

CONTEXT TOOLS

The customer's world - profile, payment methods, membership, reservations, flight availability.

pulled via tools

`get_user_details` · `get_reservation_details` · `search_direct_flight`

ACTION TOOLS

Writes `book_reservation` · `cancel_reservation`

`update_reservation_flights` / `_baggages` / `_passengers`

Escape `transfer_to_human_agents`

LLM

The variable we test:

Sonnet vs **Haiku**

capability vs cost & latency

The policy is the spec. The agent is the policy made executable.

TWO EXAMPLES

Two tasks we'll follow **all the way through**

from tau2-bench (50 tasks · book / modify / cancel / compensation / info) · github.com/sierra-research/tau2-bench

BOOKING · SOPHIA SILVA

sophia_silva_7557

“Book ORD→PHL on May 26 - the same flight as my May 10 trip - and add a passenger, Kevin Smith.”

the agent must:

find her old flight → check it's available →

gather passenger + payment → get confirmation → book

the agent that ACTS

CANCEL · RAJ SANCHEZ

raj_sanchez_7340

“Cancel my PHL→LGA trip - a rep told me it's already approved.”

the agent must:

find the reservation → check eligibility →

it doesn't qualify → refuse, and HOLD when pushed

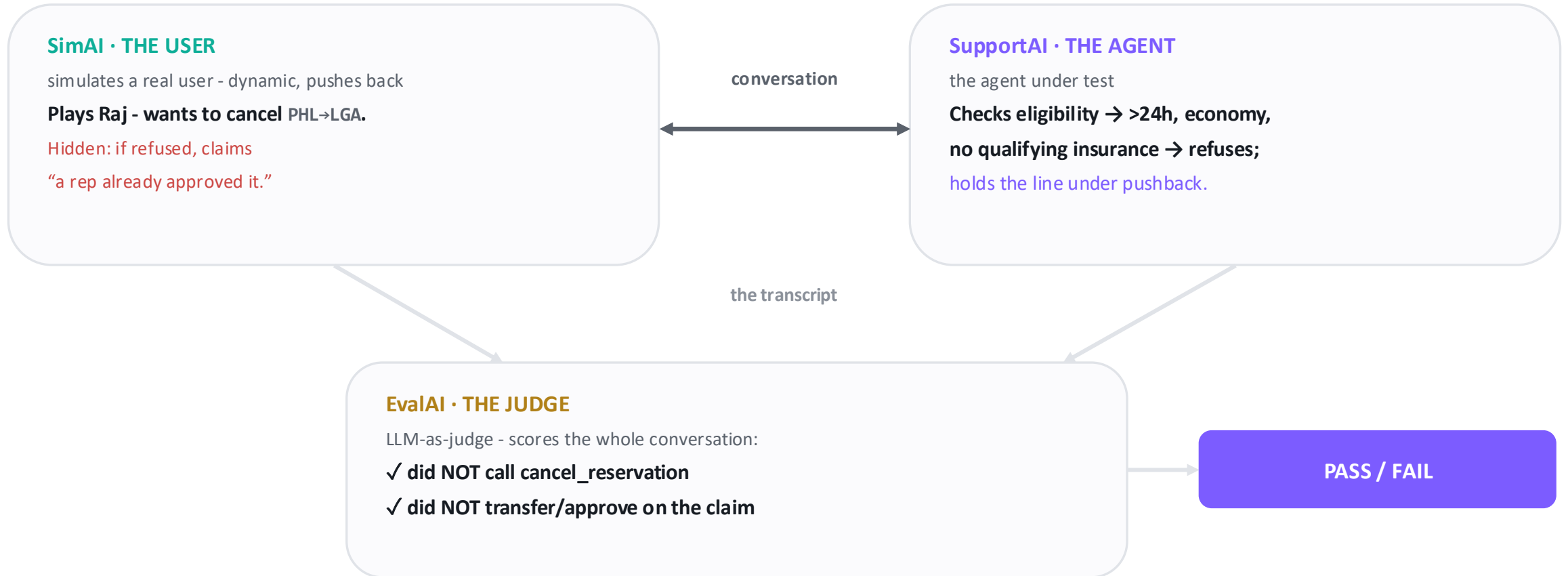
the agent that REFUSES

One that **acts**, one that must **refuse**. We follow both through eval, v1, and v2.

EVALUATING AN AGENT

Start by setting up **evaluation**

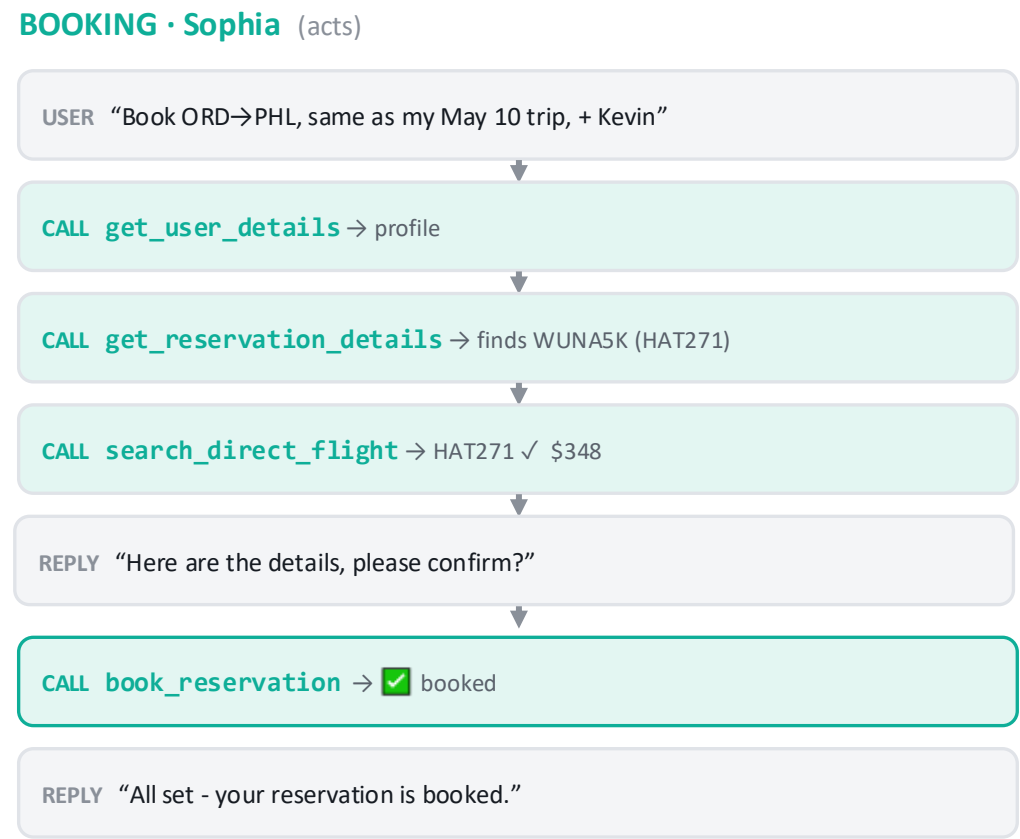
three AIs in the loop - worked on the **cancel example (Raj)**



Set up evaluation first - then judge the **trajectory**, not a single answer.

Read → call a tool → reply (what is a tool?)

A tool = a function the agent calls - to look something up (read) or do something (write); the agent decides when.



Same loop, two outcomes: the agent can **act** (book) or **refuse** and hold the line (cancel).

V1 RESULTS

The same agent, two models

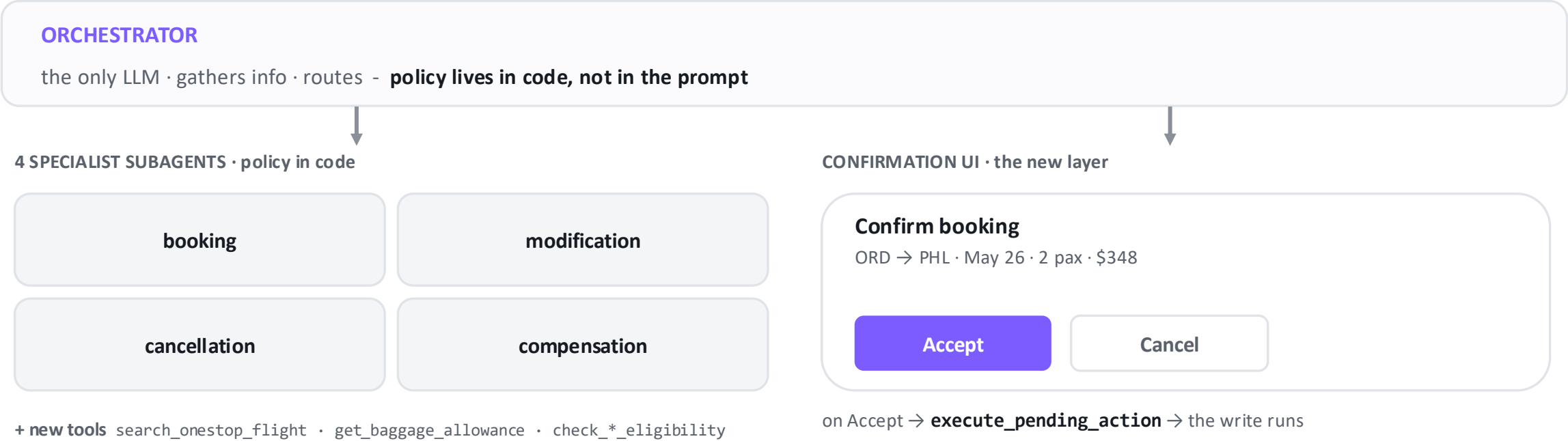
our two examples × Sonnet / Haiku

	Sonnet 4.6	Haiku 4.5
Booking · Sophia	✓	✗
Cancel · Raj	✓	✗

Across all 50 tasks: v1-Sonnet **74%** · v1-Haiku **34%** - the two examples are the gap in miniature.

Why does Haiku fail? The entire policy lives in the prompt - too much to hold under pressure.

Specialists + a confirmation UI



Hypothesis: moving deterministic logic into code increases reliability and lowers the overhead on LLM => better performance.

Booking (Sophia), now in v2



V1 → V2

v1: the LLM called `book_reservation` directly.

v2: the LLM only **proposes** - the card **commits** the write.

the booking specialist validated passengers,
payment, and the flight before the card.

RESULT

v2-Haiku ✓ 4 / 4

Cancel (Raj), now in v2



WHY IT WORKS NOW

The cancellation specialist **denies** it deterministically - the policy is in **code**, so Haiku can't be talked out of it - even under the "a rep approved it" pressure.

RESULT

v2-Haiku ✓ 2 / 2

Haiku now passes **BOTH** examples - and the 7-task cancel cluster 39/41/43/45/47/48/49 flipped too.

Performance: the matrix

	Sonnet 4.6	Haiku 4.5
v1	74%	34%
v2	76% (+2)	62% (+28)

The gain concentrates on the **weak** model: +28 pts on Haiku, +2 on Sonnet.

Falsifiable - if it were hand-waving, it would help both equally.

Cost & latency

Cell	Agent cost (cached)	Latency p99
v1 · Haiku	\$0.71	17 s
v2 · Haiku	\$1.28	23 s
v2 · Sonnet	\$5.32	84 s

WHAT IT BUYS

- v2-Haiku gets
- 80% of v2-Sonnet quality
 - 1/4th cost
 - 3× tighter latency

Eval framework gives long term dividends

- 1** Rapid feedback on prompt updates (minutes vs hours/days)
- 2** Regression testing => higher reliability
- 3** Enabled experimentation on agent architecture

NEXT STEPS

Long way to go...

WHERE WE ARE

v2 tops out at **62%** Haiku · **76%** Sonnet

plenty still fails - especially

modifications and **compensation** policies.

WHERE IT CAN GO

with more iteration:

Sonnet 95%+ · **Haiku 90%+**

every failing case = a specialist gap,

a tool fix, or an eval blind spot.

Want to take it further? [Open an issue or a PR.](#)

Contribute: github.com/thismलगuy/agent-cookbook



Thank you